

Universidad de las Fuerzas Armadas ESPE

Entorno de Desarrollo y Entrenamiento de Inteligencia Artificial

Diagnóstico de enfermedades en hojas de aguacate
mediante visión artificial

Infraestructura:	Servidor lab-dev-ultra9
Entorno:	Laboratorio L2ISW
Área:	Inteligencia Artificial / Visión Artificial
Estado:	Documento preliminar de planificación técnica

Octubre de 2026

Índice

1. Introducción	3
2. Objetivo general	3
3. Objetivos específicos	3
4. Infraestructura disponible	4
5. Arquitectura propuesta	4
6. Máquina virtual propuesta	5
7. Sistema operativo	6
8. Configuración de GPU	6
9. CUDA	7
10.Herramientas base del sistema	8
11.Entorno Python	8
12.Framework de inteligencia artificial	9
13.Paquetes iniciales	10
14.Modelo inicial	10
15.Transferencia de aprendizaje	11
16.Datos para entrenamiento	12
16.1. Fase inicial: datasets públicos	12
16.2. Fase posterior: dataset propio	12
17.Organización de los datos	13
18.Separación del dataset	13

19.Pipeline de entrenamiento	14
20.Data augmentation	15
21.Registro de experimentos	15
22.Evaluación del modelo	16
23.Selección del modelo final	17
24.Optimización para dispositivos móviles	17
25.Arquitectura de la aplicación móvil	18
26.Separación entre IA y conocimiento agronómico	19
27.Flujo de trabajo remoto	20
28.Estrategia de desarrollo	20
28.1. Etapa 1: Preparación del entorno	20
28.2. Etapa 2: Aprendizaje y validación del pipeline	21
28.3. Etapa 3: Primer modelo de aguacate	21
28.4. Etapa 4: Fine-tuning y experimentación	21
28.5. Etapa 5: Comparación de arquitecturas	21
28.6. Etapa 6: Optimización móvil	21
28.7. Etapa 7: Incorporación del dataset propio	21
29.Evolución prevista del sistema	21
30.Stack tecnológico preliminar	23
31.Conclusiones preliminares	23

1. Introducción

El presente documento define de manera preliminar el entorno tecnológico, las herramientas y el proceso de trabajo propuesto para el desarrollo de un sistema de inteligencia artificial orientado a la identificación de enfermedades y alteraciones visibles en hojas de aguacate mediante imágenes.

El proyecto contempla inicialmente el uso de conjuntos de datos públicos para construir, entrenar y evaluar los primeros modelos. Posteriormente, se prevé incorporar un conjunto de datos propio obtenido directamente en condiciones reales de campo.

El modelo resultante estará orientado a su utilización en dispositivos móviles. Por esta razón, el proceso de selección del modelo no considerará únicamente su precisión, sino también factores como tamaño, velocidad de inferencia, consumo de memoria, capacidad de procesamiento sin conexión a Internet y compatibilidad con dispositivos de recursos limitados.

2. Objetivo general

Diseñar e implementar un entorno de desarrollo y entrenamiento de inteligencia artificial que permita construir un modelo de visión artificial capaz de identificar enfermedades y alteraciones visibles en hojas de aguacate a partir de fotografías.

El sistema deberá estar preparado para evolucionar desde el uso inicial de conjuntos de datos públicos hasta la incorporación de información obtenida directamente en campo.

3. Objetivos específicos

- Configurar un entorno Linux estable para entrenamiento de modelos de inteligencia artificial mediante GPU.
- Aprovechar la GPU NVIDIA GeForce RTX 5070 Ti disponible en el servidor mediante virtualización y GPU passthrough.
- Implementar un entorno reproducible basado en Python y PyTorch.
- Construir un pipeline para preparación, entrenamiento, validación y evaluación de modelos de clasificación de imágenes.
- Utilizar inicialmente datasets públicos relacionados con enfermedades del aguacate.
- Utilizar transferencia de aprendizaje sobre modelos preentrenados para reducir los requerimientos de datos y tiempo de entrenamiento.
- Evaluar arquitecturas ligeras adecuadas para dispositivos móviles.
- Optimizar posteriormente el modelo seleccionado para su ejecución local en teléfonos móviles.

- Incorporar posteriormente un dataset propio obtenido en condiciones reales de producción agrícola.

4. Infraestructura disponible

El servidor disponible para el proyecto corresponde al equipo `lab-dev-ultra9`, utilizado como servidor de virtualización del laboratorio L2ISW.

Cuadro 1: Componentes principales del servidor	
Componente	Especificación
Procesador	Intel Core Ultra 9 285
Sockets	1
Núcleos físicos	24
Hilos lógicos	24
Hilos por núcleo	1
Frecuencia mínima	800 MHz
Frecuencia máxima	5.8 GHz
Virtualización	Intel VT-x
Memoria RAM	64 GB DDR5 CL32
GPU	NVIDIA GeForce RTX 5070 Ti
Almacenamiento	SSD NVMe de 2 TB
Placa base	ASUS
Hipervisor	Proxmox VE

La infraestructura disponible proporciona capacidad suficiente para realizar transferencia de aprendizaje, fine-tuning y experimentación con diferentes arquitecturas de visión artificial.

5. Arquitectura propuesta

Debido a que el equipo funciona como servidor de virtualización mediante Proxmox, se propone mantener separado el entorno de inteligencia artificial del sistema anfitrión.

La arquitectura general será:

```
Hardware físico
|
+-- Intel Core Ultra 9 285
+-- 64 GB RAM
```

```
+-- NVIDIA GeForce RTX 5070 Ti
+-- SSD NVMe 2 TB
|
v
Proxmox VE
|
v
Máquina virtual dedicada para IA
|
+-- Ubuntu Server 24.04 LTS
+-- CPU virtual asignada
+-- RAM asignada
+-- almacenamiento asignado
+-- RTX 5070 Ti mediante PCIe passthrough
|
v
Driver NVIDIA
|
v
CUDA
|
v
PyTorch
|
v
Entrenamiento de modelos
```

Esta separación permite conservar Proxmox como plataforma de administración y virtualización, mientras el trabajo relacionado con inteligencia artificial permanece contenido dentro de una máquina virtual específica.

6. Máquina virtual propuesta

Como configuración inicial se propone:

Cuadro 2: Configuración inicial propuesta para la VM de IA

Recurso	Configuración inicial
Nombre	avocado-ai
Sistema operativo	Ubuntu Server 24.04 LTS x86-64
CPU	12-16 vCPU
RAM	32 GB
Almacenamiento	Aproximadamente 500 GB
GPU	NVIDIA GeForce RTX 5070 Ti mediante PCIe passthrough
Acceso	SSH mediante red local

Los recursos indicados constituyen una configuración inicial y podrán modificarse posteriormente según el consumo real del proyecto y las demás cargas existentes en el servidor.

7. Sistema operativo

Se propone utilizar:

Ubuntu Server 24.04 LTS

La utilización de una versión LTS proporciona un entorno con soporte prolongado y adecuado para infraestructura que debe permanecer estable durante el desarrollo del proyecto.

Ubuntu posee además un amplio ecosistema para herramientas relacionadas con NVIDIA, CUDA, Python y PyTorch.

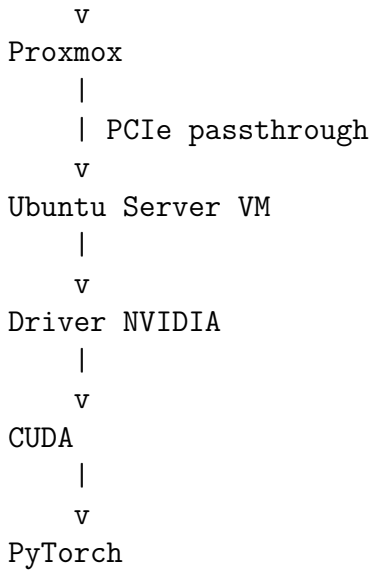
Se utilizará la edición Server debido a que el equipo será administrado principalmente mediante acceso remoto y no requiere un entorno gráfico de escritorio para realizar los entrenamientos.

8. Configuración de GPU

La GPU NVIDIA GeForce RTX 5070 Ti será asignada a la máquina virtual mediante PCIe/GPU passthrough.

El flujo esperado será:

RTX 5070 Ti
|



Una vez realizada la asignación, dentro de Ubuntu deberá comprobarse que el dispositivo es visible mediante:

```
lspci | grep -i nvidia
```

Después de instalar correctamente el controlador NVIDIA, se verificará la comunicación entre el sistema operativo y la GPU mediante:

```
nvidia-smi
```

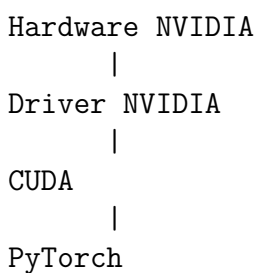
La versión exacta del controlador se seleccionará durante la implementación de acuerdo con la GPU instalada, la versión de CUDA seleccionada y la matriz de compatibilidad correspondiente.

9. CUDA

CUDA permitirá utilizar la GPU NVIDIA para las operaciones de cómputo necesarias durante el entrenamiento de las redes neuronales.

No se establece en esta etapa una versión fija de CUDA. La versión será seleccionada conjuntamente con la versión de PyTorch y el controlador NVIDIA para garantizar compatibilidad entre los componentes.

La pila deberá mantenerse conceptualmente de la siguiente manera:



| Modelo de IA

Se evitará mezclar métodos de instalación incompatibles o utilizar versiones antiguas únicamente por aparecer en tutoriales previos.

10. Herramientas base del sistema

Dentro de Ubuntu Server se requerirán inicialmente herramientas como:

- Git;
- curl;
- wget;
- build-essential;
- Python 3;
- pip;
- venv;
- OpenSSH Server;
- tmux;
- htop;
- unzip.

Git será utilizado para el control de versiones del código.

OpenSSH permitirá administrar y utilizar el servidor desde una computadora externa.

La herramienta `tmux` permitirá mantener sesiones y procesos de entrenamiento ejecutándose incluso cuando el usuario cierre temporalmente su conexión SSH.

11. Entorno Python

Las dependencias del proyecto no serán instaladas directamente en el entorno global del sistema operativo.

Se utilizará un entorno virtual de Python dedicado al proyecto.

La estructura conceptual será:

```
Ubuntu Server
|
+-- Python
   |
   +-- Entorno virtual avocado-ai
       |
       +-- PyTorch
       +-- torchvision
       +-- NumPy
       +-- Pandas
       +-- scikit-learn
       +-- OpenCV
       +-- Pillow
       +-- Matplotlib
       +-- JupyterLab
```

Esto permitirá controlar las versiones de las dependencias y mejorar la reproducibilidad del proyecto.

12. Framework de inteligencia artificial

El framework principal propuesto es:

PyTorch

PyTorch será utilizado para:

- manejo de tensores;
- utilización de GPU;
- definición y modificación de redes neuronales;
- carga y procesamiento de datasets;
- entrenamiento;
- cálculo de gradientes;
- optimización;
- evaluación;
- almacenamiento de checkpoints;
- exportación posterior del modelo.

Se utilizará adicionalmente `torchvision` para procesamiento de imágenes, transformaciones y acceso a arquitecturas de visión artificial predefinidas.

13. Paquetes iniciales

El entorno de entrenamiento requerirá inicialmente:

Cuadro 3: Principales librerías propuestas

Librería	Utilización
PyTorch	Desarrollo y entrenamiento de redes neuronales
torchvision	Modelos y procesamiento de imágenes
NumPy	Operaciones numéricas
Pandas	Manejo y análisis de metadatos
scikit-learn	Métricas y herramientas de evaluación
Pillow	Lectura y manipulación de imágenes
OpenCV	Procesamiento de imágenes
Matplotlib	Visualización de resultados
JupyterLab	Exploración y experimentación inicial

Posteriormente podrán incorporarse herramientas como:

- Albumentations;
- TensorBoard;
- ONNX;
- ONNX Runtime;
- MLflow;
- Optuna;
- DVC.

Estas herramientas serán añadidas únicamente cuando exista una necesidad concreta dentro del proyecto.

14. Modelo inicial

Como primer modelo de referencia se propone utilizar:

MobileNetV3 Large

MobileNet pertenece a una familia de arquitecturas diseñadas teniendo en cuenta dispositivos con recursos computacionales limitados, por lo que constituye un candidato apropiado para un sistema cuyo destino final será un teléfono móvil.

Sin embargo, MobileNetV3 no se considera desde esta etapa como el modelo definitivo.

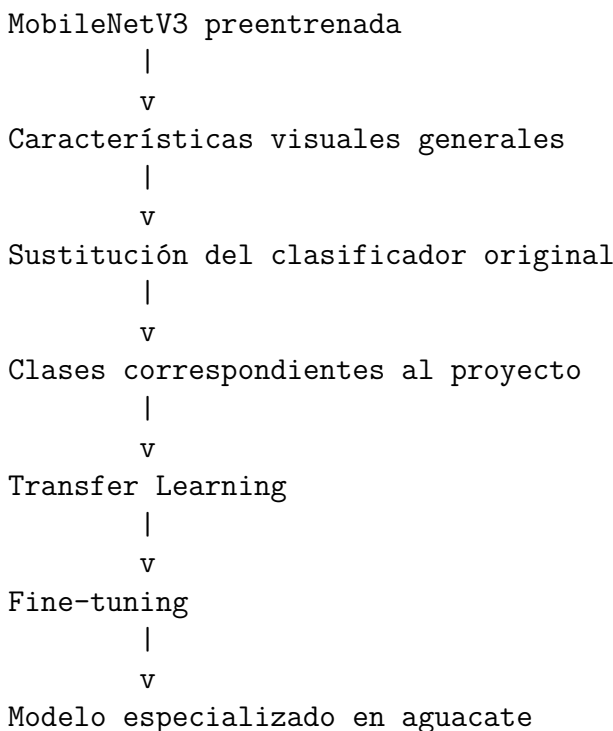
Funcionará como *baseline* contra el cual podrán compararse posteriormente otras arquitecturas ligeras.

15. Transferencia de aprendizaje

El modelo destinado al sistema final no será entrenado inicialmente desde pesos aleatorios.

Se utilizará transferencia de aprendizaje (*transfer learning*) a partir de pesos preentrenados.

El procedimiento general será:



Durante una primera etapa podrán congelarse determinadas capas del modelo y entrenarse principalmente las capas de clasificación.

Posteriormente se realizará *fine-tuning*, permitiendo que parte de las capas previamente entrenadas adapten sus parámetros a las características específicas de las hojas y enfermedades del aguacate.

16. Datos para entrenamiento

El proyecto contempla dos etapas principales de obtención de datos.

16.1. Fase inicial: datasets públicos

Mientras no se disponga de acceso al entorno agrícola, se utilizarán conjuntos de datos públicos para desarrollar el pipeline completo.

Antes de utilizar cada dataset se analizarán aspectos como:

- cantidad de imágenes;
- clases disponibles;
- distribución de imágenes por clase;
- resolución;
- calidad;
- procedencia;
- método de etiquetado;
- presencia de duplicados;
- posibles errores de clasificación;
- licencia y condiciones de utilización.

16.2. Fase posterior: dataset propio

Posteriormente se prevé disponer de acceso a aproximadamente 900 árboles de aguacate.

Esto permitirá recopilar fotografías bajo condiciones reales y construir progresivamente un dataset especializado.

La información deberá incluir metadatos que permitan conocer, cuando sea posible:

- árbol de procedencia;
- fecha;
- sector o ubicación dentro de la finca;
- variedad;
- parte de la planta;
- condición observada;

- diagnóstico;
- nivel de certeza del diagnóstico;
- severidad;
- observaciones adicionales.

17. Organización de los datos

Se propone mantener una estructura similar a:

```
data/
|
+-- raw/
|   +-- Datos originales sin modificar
|
+-- interim/
|   +-- Datos parcialmente procesados
|
+-- processed/
|   +-- Datos preparados para entrenamiento
|
+-- splits/
|   +-- train.csv
|   +-- val.csv
|   +-- test.csv
```

Los datos originales almacenados en **raw** deberán conservarse sin modificaciones para mantener una fuente original reproducible.

18. Separación del dataset

Los datos utilizados para el desarrollo del modelo serán separados conceptualmente en:

- conjunto de entrenamiento;
- conjunto de validación;
- conjunto de prueba.

El conjunto de entrenamiento será utilizado para actualizar los parámetros del modelo.

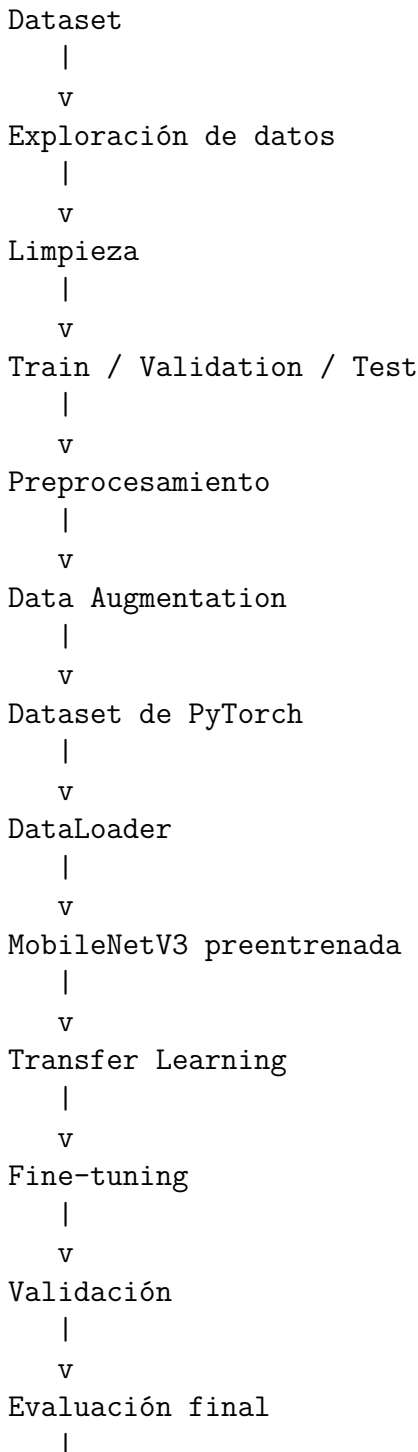
El conjunto de validación permitirá evaluar las decisiones realizadas durante el desarrollo.

El conjunto de prueba permanecerá separado y será utilizado para la evaluación final.

Cuando se disponga del dataset propio, se deberá evitar que fotografías muy similares del mismo árbol aparezcan simultáneamente en entrenamiento y prueba, ya que esto podría producir estimaciones artificialmente optimistas del rendimiento.

19. Pipeline de entrenamiento

El proceso general será:



v

Análisis de errores

El pipeline será diseñado para permitir la repetición de experimentos bajo condiciones controladas.

20. Data augmentation

Debido a que las imágenes finales serán obtenidas mediante teléfonos móviles en condiciones reales, será necesario aumentar la robustez del modelo frente a variaciones de captura.

Durante el entrenamiento podrán aplicarse transformaciones controladas como:

- rotaciones;
- recortes;
- cambios moderados de brillo;
- cambios moderados de contraste;
- escalado;
- desenfoque moderado;
- otras transformaciones compatibles con condiciones reales.

Las transformaciones deberán representar situaciones plausibles y no modificar artificialmente características importantes para el diagnóstico.

21. Registro de experimentos

Cada entrenamiento deberá mantener información suficiente para poder ser reproducido posteriormente.

Una estructura posible será:

```
experiments/  
|  
+-- experiment_001/  
|   +-- config.yaml  
|   +-- metrics.json  
|   +-- training.log  
|   +-- confusion_matrix.png  
|   +-- curves.png
```

```
|   +-- checkpoint.pt
|
+-- experiment_002/
    +-- ...
```

Cada experimento deberá registrar como mínimo:

- arquitectura utilizada;
- versión del dataset;
- tamaño de imagen;
- tamaño del batch;
- learning rate;
- número de epochs;
- optimizador;
- transformaciones;
- semilla utilizada;
- versión de PyTorch;
- GPU utilizada;
- métricas obtenidas.

22. Evaluación del modelo

La evaluación no se limitará únicamente a la exactitud (*accuracy*).

Se analizarán métricas como:

- accuracy;
- precision;
- recall;
- F1-score;
- F1-score por clase;
- recall por enfermedad;
- matriz de confusión.

En etapas posteriores también se analizarán métricas relacionadas con el dispositivo móvil:

- tamaño del modelo;
- utilización de memoria;
- tiempo de inferencia;
- comportamiento en diferentes teléfonos;
- impacto de la optimización sobre la precisión.

23. Selección del modelo final

MobileNetV3 será utilizado como primer modelo de referencia, pero la arquitectura definitiva será seleccionada experimentalmente.

La comparación deberá considerar simultáneamente:

- capacidad de clasificación;
- recall por enfermedad;
- F1-score;
- tamaño del modelo;
- velocidad de inferencia;
- utilización de RAM;
- compatibilidad móvil.

De esta forma, el modelo seleccionado no será necesariamente aquel con la mayor precisión absoluta, sino aquel que proporcione el mejor equilibrio para el contexto real de utilización.

24. Optimización para dispositivos móviles

Una vez seleccionado un modelo satisfactorio, se estudiarán técnicas de optimización.

El proceso general será:

Modelo entrenado

FP32

|

```

v
Optimización
|
+-- FP16
|
+-- INT8
|
v
Modelo optimizado
|
v
Benchmark en dispositivo móvil

```

Se comparará el modelo original con sus versiones optimizadas para determinar los cambios producidos en:

- precisión;
- F1-score;
- tamaño;
- memoria;
- velocidad.

25. Arquitectura de la aplicación móvil

El sistema final se plantea inicialmente con inferencia local.

```

Telefono movil
|
v
Camara
|
v
Imagen
|
v
Modelo de vision artificial
|
v
Prediccion
|
+-- Enfermedad
|
+-- Nivel de confianza

```

```

      |
      v
Base de conocimiento
      |
      +-- Descripcion
      +-- Sintomas
      +-- Prevencion
      +-- Manejo
      +-- Tratamiento
      +-- Fuentes

```

El objetivo será permitir que la identificación básica pueda realizarse sin conexión permanente a Internet.

26. Separación entre IA y conocimiento agronómico

La red neuronal tendrá como responsabilidad principal analizar la imagen y producir una clasificación.

La información relacionada con descripción, síntomas, prevención, manejo y tratamiento será administrada independientemente mediante una base de conocimiento.

Por ejemplo:

```

Imagen
      |
      v
Modelo
      |
      v
disease_id = 03
      |
      v
Base de conocimiento
      |
      +-- Nombre
      +-- Descripcion
      +-- Sintomas
      +-- Prevencion
      +-- Manejo
      +-- Tratamiento
      +-- Referencias

```

Esta separación permitirá actualizar información agronómica sin necesidad de volver a entrenar la red neuronal.

27. Flujo de trabajo remoto

El desarrollo podrá realizarse desde una computadora personal mediante SSH.

La arquitectura será:

```
Laptop del desarrollador
|
+-- Visual Studio Code
+-- Git
+-- SSH
|
| Conexión de red
v
VM Ubuntu Server
|
+-- Código
+-- Datasets
+-- Experimentos
+-- Python
+-- PyTorch
|
v
CUDA
|
v
RTX 5070 Ti
```

De esta forma, la computadora personal funcionará principalmente como estación de desarrollo, mientras el procesamiento intensivo será realizado por el servidor.

28. Estrategia de desarrollo

El proyecto se dividirá inicialmente en las siguientes etapas:

28.1. Etapa 1: Preparación del entorno

- creación de la máquina virtual;
- instalación de Ubuntu Server;
- configuración de GPU passthrough;
- instalación del controlador NVIDIA;
- configuración de CUDA;

- instalación de Python;
- instalación de PyTorch;
- verificación de acceso a GPU.

28.2. Etapa 2: Aprendizaje y validación del pipeline

Se desarrollará inicialmente una red neuronal pequeña con fines didácticos para comprender el ciclo completo de entrenamiento, incluyendo forward propagation, función de pérdida, backpropagation, gradientes y optimización.

28.3. Etapa 3: Primer modelo de aguacate

Se seleccionará un dataset público y se utilizará MobileNetV3 con transferencia de aprendizaje para obtener un primer modelo funcional.

28.4. Etapa 4: Fine-tuning y experimentación

Se realizarán experimentos modificando hiperparámetros, transformaciones y estrategias de fine-tuning.

28.5. Etapa 5: Comparación de arquitecturas

MobileNetV3 será comparada con otras arquitecturas ligeras para determinar cuál ofrece el mejor equilibrio entre rendimiento y requerimientos computacionales.

28.6. Etapa 6: Optimización móvil

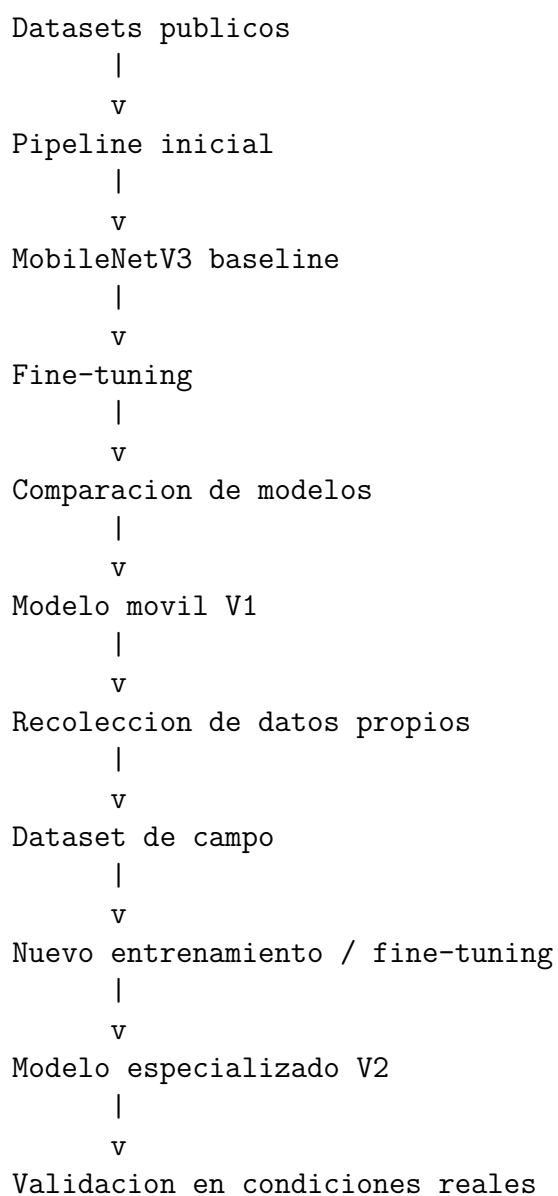
El modelo seleccionado será optimizado y evaluado en dispositivos móviles reales.

28.7. Etapa 7: Incorporación del dataset propio

Una vez disponible el acceso a los árboles de aguacate, se iniciará la recolección sistemática de imágenes y metadatos para desarrollar una versión especializada del modelo.

29. Evolución prevista del sistema

La evolución general esperada será:



30. Stack tecnológico preliminar

Cuadro 4: Stack tecnológico propuesto

Capa	Tecnología
Hipervisor	Proxmox VE
Sistema operativo	Ubuntu Server 24.04 LTS
GPU	NVIDIA GeForce RTX 5070 Ti
Acceso GPU	PCIe passthrough
Controlador	NVIDIA Driver compatible
Computación GPU	CUDA
Lenguaje	Python
Entorno Python	venv
Framework ML	PyTorch
Visión	torchvision / OpenCV / Pillow
Análisis	NumPy / Pandas
Evaluación	scikit-learn
Visualización	Matplotlib
Modelo baseline	MobileNetV3 Large
Metodología	Transfer Learning + Fine-tuning
Control de versiones	Git
Acceso remoto	SSH
Destino	Aplicación móvil Android
Inferencia objetivo	Local / offline
Optimización prevista	FP16 / INT8

31. Conclusiones preliminares

La infraestructura disponible resulta adecuada para el desarrollo del proyecto. La combinación del procesador Intel Core Ultra 9 285, 64 GB de memoria RAM, almacenamiento NVMe y una GPU NVIDIA GeForce RTX 5070 Ti proporciona capacidad suficiente para realizar experimentación y entrenamiento de modelos de visión artificial.

Se propone mantener el entorno de inteligencia artificial aislado en una máquina virtual Ubuntu Server 24.04 LTS dentro de Proxmox, asignando la GPU mediante PCIe passthrough.

PyTorch será utilizado como framework principal de desarrollo y MobileNetV3 Large funcionará inicialmente como arquitectura baseline debido a su orientación hacia escenarios con recursos computacionales limitados.

El desarrollo comenzará utilizando datasets públicos, permitiendo construir y validar todo el pipeline antes de disponer de datos propios.

Posteriormente, la posibilidad de recopilar información directamente de aproximadamente 900 árboles de aguacate permitirá desarrollar un dataset propio bajo condiciones reales de campo. Este dataset podrá utilizarse para adaptar y evaluar el modelo en el contexto específico en el que finalmente será utilizado.

La arquitectura definitiva no será seleccionada únicamente por su accuracy, sino mediante una comparación que considere simultáneamente calidad de clasificación, recall por enfermedad, F1-score, tamaño del modelo, consumo de memoria y velocidad de inferencia en dispositivos móviles.

Finalmente, el modelo de visión artificial se mantendrá separado de la base de conocimiento agronómico. De esta manera será posible actualizar información sobre síntomas, prevención, manejo y tratamientos sin necesidad de volver a entrenar la red neuronal.